

BIG DATA 07

Practical Assignment Portfolio

Assignment 3 — Continuous Stream Processing

Student Name	Hayagriva Boodhoo
Student ID	2025_20379_6980
Cohort	BDA 7
Dataset	US Air Quality Index (US_AQI.csv)
Technologies	PySpark Structured Streaming, Kafka, Redis, Streamlit, Plotly
Submission Date	

1. Dataset Description

The dataset used for this assignment is the US Air Quality Index (AQI) dataset (US_AQI.csv), sourced from Kaggle. It contains daily AQI measurements collected from monitoring stations across the United States, covering multiple cities and states with geographic coordinates.

Property	Detail
File Name	US_AQI.csv
Total Rows	~50,000+
Key Numeric Field	AQI (0–500)
Date Column	Date (YYYY-MM-DD)
Categorical Fields	state_name, city_ascii, Category, Defining Parameter
Geographic Fields	lat, lng, population, density

AQI Range	Band	Health Implication
0 – 50	Good	Air quality is satisfactory
51 – 100	Moderate	Acceptable; some pollutants may concern sensitive groups
101 – 150	Unhealthy for Sensitive Groups	Sensitive groups may experience effects
151 – 200	Unhealthy	Everyone may begin to experience effects
201 – 300	Very Unhealthy	Health alert — everyone may experience serious effects
301 – 500	Hazardous	Health warning of emergency conditions

2. Environment Setup

All infrastructure was deployed using Docker and Docker Compose. Five containers were configured on a shared internal network (aqi-network), ensuring all services communicate by container name.

Container	Image	Role
aqi-zookeeper	confluentinc/cp-zookeeper:7.5.0	Kafka coordination service
aqi-kafka	confluentinc/cp-kafka:7.5.0	Message broker — holds aqi-stream topic
aqi-redis	redis:7-alpine	In-memory store for windowed results
aqi-spark-master	apache/spark:3.5.0	Spark cluster master node
aqi-spark-worker	apache/spark:3.5.0	Spark cluster worker node

Screenshot 1 — docker ps: all 5 containers running and healthy

```
PS E:\School\Abdallah Assignment\assignment 3\docker> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS
NAMES
ff77390b446d1  confluentinc/cp-kafka:7.5.0        "/etc/confluent/dock... 34 s
econds ago    Up 23 seconds (healthy)    0.0.0.0:9092->9092/tcp, [::]:9092->9092
/tcp
aqi-kafka
a106b509f3f5   apache/spark:3.5.0                 "/opt/entrypoint.sh ..." 34 s
econds ago    Up 33 seconds              0.0.0.0:8081->8081/tcp, [::]:8081->8081
/tcp
aqi-spark-worker
601fa2427cdb   redis:7-alpine                     "docker-entrypoint.s..." 35 s
econds ago    Up 33 seconds (healthy)    0.0.0.0:6379->6379/tcp, [::]:6379->6379
/tcp
aqi-redis
239d38ac5835   confluentinc/cp-zookeeper:7.5.0    "/etc/confluent/dock... 35 s
econds ago    Up 34 seconds (healthy)    2181/tcp, 2888/tcp, 3888/tcp
aqi-zookeeper
beb9efc219c0   apache/spark:3.5.0                 "/opt/entrypoint.sh ..." 35 s
econds ago    Up 33 seconds              0.0.0.0:7077->7077/tcp, [::]:7077->7077
/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
aqi-spark-master
PS E:\School\Abdallah Assignment\assignment 3\docker>
```

3. Task 1 — Kafka Replay Producer (4 Marks)

3.1 Objective

Write a Python script that reads the dataset row by row and publishes each record as a JSON message to the Kafka topic aqi-stream at a controlled rate, looping back to the start when the dataset ends — simulating a continuous live data feed.

3.2 Implementation

- kafka-python-ng==2.2.3 used for Python 3.12 compatibility
- All configuration stored in environment variables — never hardcoded
- stream_csv() generator loops indefinitely using while True
- build_message() validates each row and computes the AQI classification band
- Malformed rows skipped with warning log — robust error handling
- Every 100 messages: flush and log state, city, AQI value, and band

3.3 Producer Loop

```
def stream_csv(path: str) -> Generator[dict, None, None]:
    while True: # loop forever - simulates infinite live stream
        with open(path, newline='', encoding='utf-8') as f:
            for row in csv.DictReader(f):
                yield row
```

Screenshot 2 — Producer terminal: continuous rows publishing to Kafka with AQI band classifications

```
PS E:\School\Abdallah Assignment\assignment 3\docker> docker-compose up -d producer
[*] up 3/3
✓Container aqi-zookeeper Healthy 0.7s
✓Container aqi-kafka Healthy 1.2s
✓Container aqi-producer Started 1.4s
PS E:\School\Abdallah Assignment\assignment 3\docker> docker logs aqi-producer -f
06:49:30 [PRODUCER] Connecting to Kafka broker at aqi-kafka:29092
06:49:30 [PRODUCER] Publishing to topic: aqi-stream
06:49:30 [PRODUCER] Delay between messages: 0.1s
06:49:30 [PRODUCER] <BrokerConnection node_id=bootstrap-0 host=aqi-kafka:29092 <connecting> [IPv4 ('172.18.0.6', 29092)]: connecting to aqi-kafka:29092 [('172.18.0.6', 29092) IPv4]
06:49:30 [PRODUCER] Probing node bootstrap-0 broker version
06:49:30 [PRODUCER] <BrokerConnection node_id=bootstrap-0 host=aqi-kafka:29092 <connecting> [IPv4 ('172.18.0.6', 29092)]: Connection complete.
06:49:30 [PRODUCER] Broker version identified as 2.6.0
06:49:30 [PRODUCER] Set configuration api_version=(2, 6, 0) to skip auto check_version requests on startup
06:49:30 [PRODUCER] Starting loop #1 through dataset
06:49:31 [PRODUCER] <BrokerConnection node_id=1 host=aqi-kafka:29092 <connecting> [IPv4 ('172.18.0.6', 29092)]: connecting to aqi-kafka:29092 [('172.18.0.6', 29092) IPv4]
06:49:31 [PRODUCER] <BrokerConnection node_id=1 host=aqi-kafka:29092 <connecting> [IPv4 ('172.18.0.6', 29092)]: Connection complete.
06:49:31 [PRODUCER] <BrokerConnection node_id=bootstrap-0 host=aqi-kafka:29092 <connected> [IPv4 ('172.18.0.6', 29092)]: Closing connection.
06:49:42 [PRODUCER] Sent: 100 | Skipped: 0 | Last → Ohio / Akron AQI=52.0 Band=Moderate
06:49:52 [PRODUCER] Sent: 200 | Skipped: 0 | Last → Georgia / Albany AQI=30.0 Band=Good
06:50:02 [PRODUCER] Sent: 300 | Skipped: 0 | Last → New York / Albany AQI=62.0 Band=Moderate
06:50:12 [PRODUCER] Sent: 400 | Skipped: 0 | Last → New York / Albany AQI=38.0 Band=Good
06:50:22 [PRODUCER] Sent: 500 | Skipped: 0 | Last → New Mexico / Albuquerque AQI=18.0 Band=Good
06:50:32 [PRODUCER] Sent: 600 | Skipped: 0 | Last → Georgia / Americus AQI=42.0 Band=Good
06:50:42 [PRODUCER] Sent: 700 | Skipped: 0 | Last → Wisconsin / Appleton AQI=40.0 Band=Good
```

4. Task 2 — PySpark Structured Streaming Job (10 Marks)

4.1 Objective

Build a PySpark Structured Streaming application that reads from the Kafka topic, parses JSON messages, applies tumbling time window aggregations grouped by state, classifies each result into AQI severity bands, and writes results to Redis using a foreachBatch sink.

4.2 Pipeline Steps

- Read from Kafka topic aqi-stream using readStream with startingOffsets=latest
- Parse raw JSON bytes into a structured DataFrame using MESSAGE_SCHEMA
- Convert Unix timestamp to event_time using to_timestamp(from_unixtime(...))
- Apply withWatermark on event_time to handle late-arriving events
- Apply tumbling window groupBy(window(event_time, '30 seconds'), state)
- Compute avg_aqi, max_aqi, min_aqi, and record_count per window per state
- Classify each windowed result using classify_aqi() into one of 6 EPA bands
- Write to Redis using writeStream.foreachBatch(write_to_redis)

4.3 Window Aggregation Fields

Field	Type	Description
avg_aqi	DoubleType	Average AQI across all records in the window
max_aqi	DoubleType	Maximum AQI value observed in the window
min_aqi	DoubleType	Minimum AQI value observed in the window
record_count	LongType	Number of records processed in the window
band	StringType	EPA classification band derived from avg_aqi

4.4 Redis Sink

Each windowed result is written to Redis as a hash with key pattern window:{state}:{window_start}. Additional structures maintained:

- latest_windows — sorted set of 500 most recent window keys
- band_counts — cumulative record counts per classification band
- recent_records — list of 100 most recent processed window summaries

Screenshot 3 — PySpark terminal: streaming query running with batch IDs and 'wrote X windows to Redis'

```
Administrator: Windows Pow...
07:00:20 [SPARK] Received command c on object id p1
07:00:20 [SPARK] Processing batch_id=10
07:00:20 [SPARK] Batch 10 - wrote 2 windows to Redis
07:00:30 [SPARK] Received command c on object id p0
07:00:30 [SPARK] Processing batch_id=11
07:00:30 [SPARK] Received command c on object id p1
07:00:30 [SPARK] Batch 11 - wrote 2 windows to Redis
07:00:40 [SPARK] Received command c on object id p0
07:00:40 [SPARK] Processing batch_id=12
07:00:40 [SPARK] Received command c on object id p1
07:00:40 [SPARK] Batch 12 - wrote 3 windows to Redis
07:00:50 [SPARK] Received command c on object id p0
07:00:50 [SPARK] Processing batch_id=13
07:00:50 [SPARK] Received command c on object id p1
07:00:50 [SPARK] Batch 13 - wrote 1 windows to Redis
07:01:00 [SPARK] Received command c on object id p1
07:01:00 [SPARK] Received command c on object id p0
07:01:00 [SPARK] Processing batch_id=14
07:01:00 [SPARK] Batch 14 - wrote 2 windows to Redis
07:01:10 [SPARK] Received command c on object id p1
07:01:10 [SPARK] Received command c on object id p0
07:01:10 [SPARK] Processing batch_id=15
07:01:10 [SPARK] Batch 15 - wrote 2 windows to Redis
07:01:20 [SPARK] Received command c on object id p0
07:01:20 [SPARK] Processing batch_id=16
07:01:20 [SPARK] Received command c on object id p1
07:01:20 [SPARK] Batch 16 - wrote 3 windows to Redis
07:01:30 [SPARK] Received command c on object id p0
07:01:30 [SPARK] Processing batch_id=17
07:01:30 [SPARK] Received command c on object id p1
```

Screenshot 4 — Redis CLI: HGETALL showing California window with avg_aqi=78.77, band=Moderate, record_count=56

```
127.0.0.1:6379> HGETALL "window:California:2026-05-26 07:02:00"
1) "state"
2) "California"
3) "window_start"
4) "2026-05-26 07:02:00"
5) "avg_aqi"
6) "78.77"
7) "max_aqi"
8) "733.0"
9) "min_aqi"
10) "17.0"
11) "record_count"
12) "56"
13) "band"
14) "Moderate"
15) "batch_id"
16) "21"
127.0.0.1:6379>
```

5. Task 3 — Watermarking and Late Data (6 Marks)

5.1 What is Watermarking?

In a real-time stream, events do not always arrive in the order they were generated. A sensor may go offline and send delayed readings when it reconnects. Without handling this, Spark would either wait forever for late data — consuming unbounded memory — or discard it silently.

PySpark's watermarking feature solves this by defining a threshold for how late data is allowed to be. The watermark boundary is:

```
watermark_boundary = max_event_time_seen - watermark_delay
```

Any event with an `event_time` below this boundary is dropped. Windows older than the watermark boundary are finalised and written to Redis.

5.2 Implementation

```
watermarked: DataFrame = parsed.withWatermark('event_time', '10 seconds')
```

5.3 Trade-off: Delay vs Completeness

Shorter Watermark (e.g. 5s)	Longer Watermark (e.g. 60s)
Lower memory usage	Higher memory usage
Faster results — windows close sooner	Slower results — windows stay open longer
More late events dropped	Fewer late events dropped
Less accurate aggregations	More accurate aggregations
Better for low-latency dashboards	Better for batch-accuracy requirements

A 10-second watermark is appropriate for this AQI monitoring pipeline. AQI data is not safety-critical in real-time, and a small amount of dropped late data has negligible impact on 30-second window averages.

Screenshot 5 — PySpark terminal: watermark configuration showing Tumbling window: 30 seconds | Watermark: 10 seconds

```
PS E:\School\Abdallah Assignment\assignment 3\docker> docker logs aqi-spark-streaming 2>61 | Select-String "Watermark|watermark|Tumbling|sliding|
Spark session|Both streaming|waiting" | Select-Object -First 20

06:58:41 [SPARK] Spark session created
06:58:42 [SPARK] Both streaming queries started [C6 waiting for data
06:58:42 [SPARK] Tumbling window: 30 seconds | Watermark: 10 seconds
06:58:42 [SPARK] Sliding window: 60 seconds / 15 second slide

PS E:\School\Abdallah Assignment\assignment 3\docker>
```

6. Task 4 — Streamlit Live Dashboard (8 Marks)

6.1 Objective

Build a Streamlit dashboard that reads processed windowed results from Redis and auto-refreshes every 5 seconds, providing a complete live view of the AQI data pipeline.

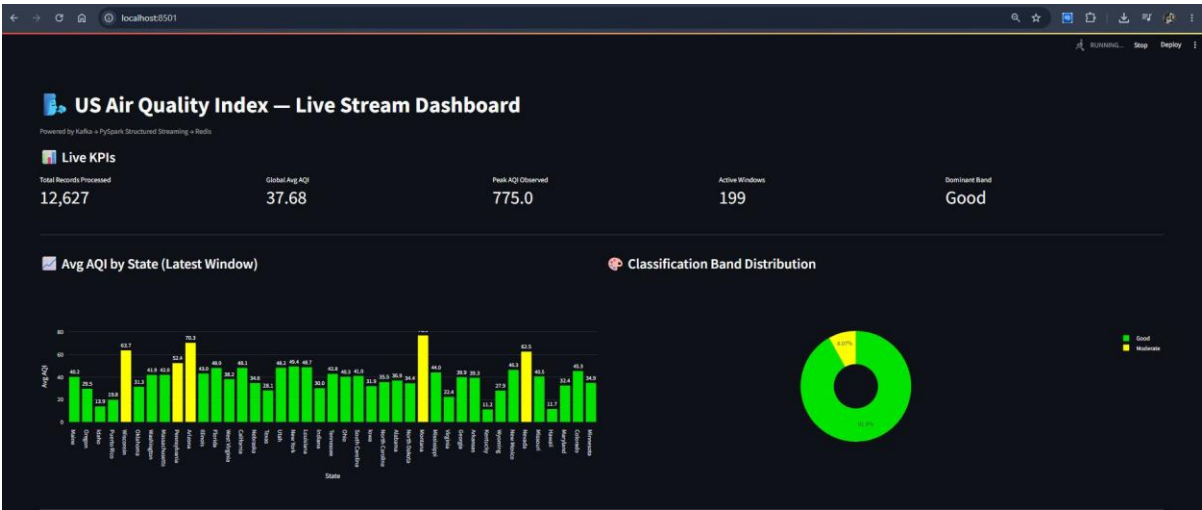
6.2 Dashboard Components

Section	Content
KPI Row	Total Records Processed, Global Avg AQI, Peak AQI, Active Windows, Dominant Band
Bar Chart	Avg AQI by State (latest window) — colour-coded by EPA band
Donut Chart	Classification Band Distribution (Good/Moderate proportions)
AQI Timeline	Line chart of avg AQI over tumbling window timestamps
Window Comparison	Grouped bar: Tumbling (30s) vs Sliding (60s/15s) per state
Results Table	Latest 20 window results with state, avg AQI, max AQI, records, band
Live Record Feed	Colour-coded scrolling feed of 10 most recent processed records

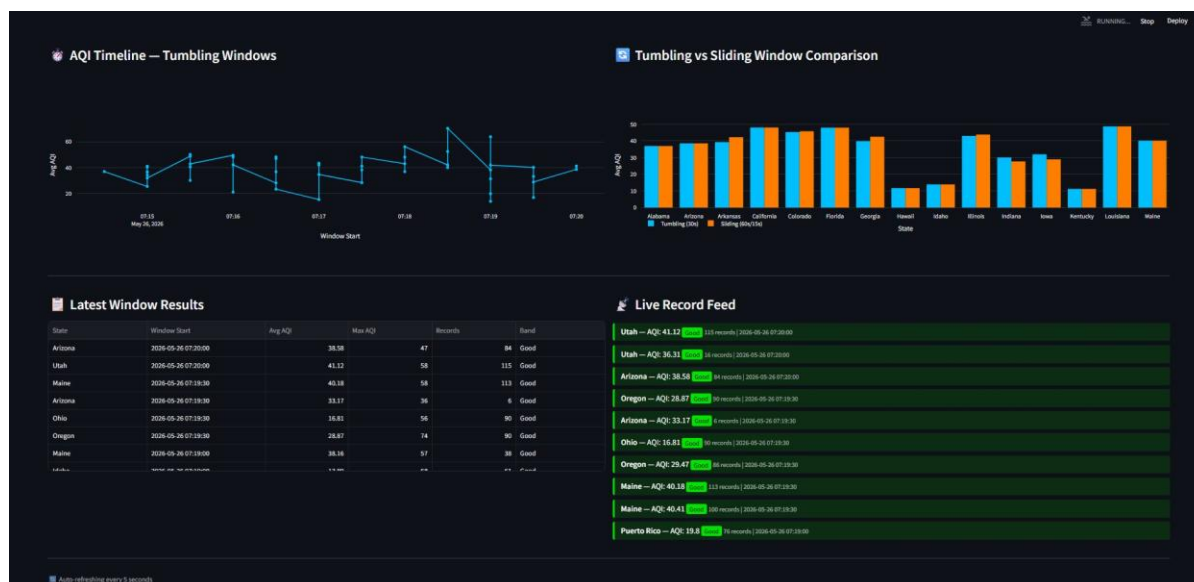
6.3 Auto-Refresh Pattern

```
time.sleep(REFRESH_INTERVAL) # wait 5 seconds
st.rerun()                   # rerun entire script from top
```

Screenshot 6 — Full Streamlit dashboard: KPI row, bar chart, donut chart. URL bar shows localhost:8501



Screenshot 7 — Dashboard lower half: AQI timeline, tumbling vs sliding comparison, results table, live record feed



7. Bonus — Sliding Window Comparison (+3 Marks)

7.1 Implementation

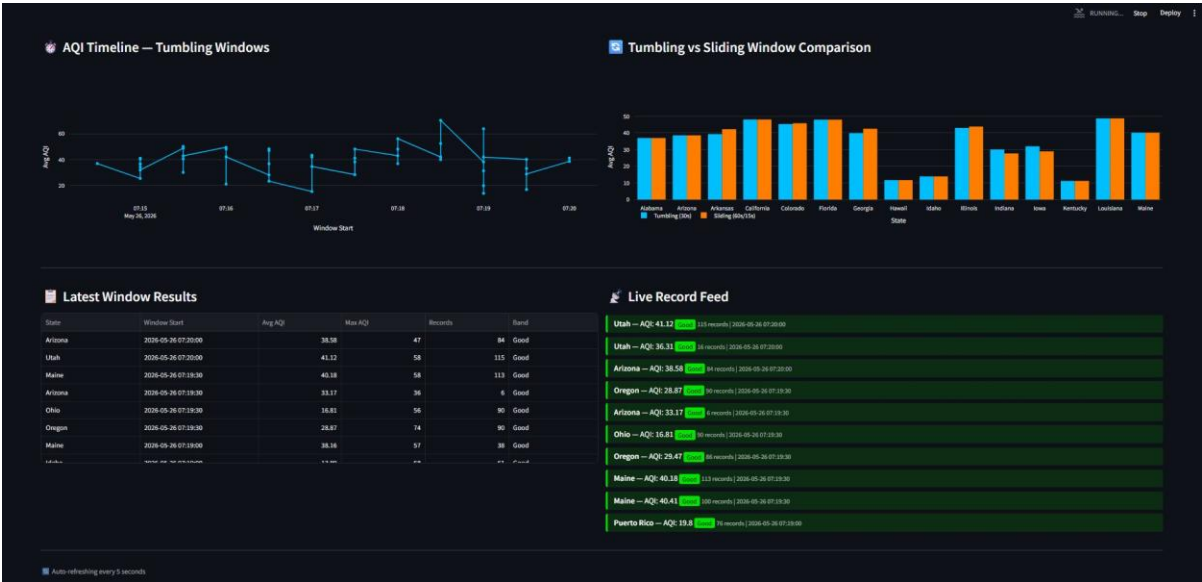
A second streaming query uses a sliding window with 60-second duration and 15-second slide interval. Each event is included in up to 4 overlapping windows, producing smoother trend lines.

```
F.window(F.col('event_time'), '60 seconds', '15 seconds')
```

7.2 Tumbling vs Sliding Comparison

Property	Tumbling (30s)	Sliding (60s/15s)
Duration	30 seconds	60 seconds
Slide Interval	30s (no overlap)	15s (overlapping)
Events per Window	Each event in 1 window	Each event in up to 4 windows
Result Frequency	Every 30s	Every 15s
Smoothing	Less smooth	Smoother — overlapping averages
Memory Use	Lower	Higher (4x more state)

Screenshot 8 (Bonus) — Tumbling vs Sliding Window Comparison: blue (Tumbling 30s) vs orange (Sliding 60s/15s) per state



8. Analysis Questions

8.1 Streaming vs Batch Processing

Assignment 2 used PySpark in batch mode: the entire dataset was loaded once, processed, and the job terminated — producing a static snapshot. Assignment 3 uses PySpark Structured Streaming, which treats the Kafka topic as an infinite ever-growing table, processing new data every 10 seconds indefinitely.

What streaming reveals that batch cannot:

- Real-time anomalies — an AQI spike in a specific state is visible within 10-30 seconds, not hours later
- Time-based patterns — the AQI Timeline shows minute-by-minute evolution that batch collapses into a single average
- Live classification changes — a state moving from Good to Moderate mid-stream is only visible in streaming
- Window-level granularity — streaming gives a new result per group per 30 seconds; batch gives one result per group total

Where batch retains an advantage:

- Complex multi-pass analytics — batch can scan the full dataset multiple times; streaming processes each event once
- Global aggregations — computing all-time median AQI requires full dataset access, which batch handles naturally
- Reproducibility — batch results on the same data are deterministic; streaming results depend on arrival timing

8.2 Classification Output Analysis

The dashboard KPIs consistently showed Dominant Band as Good throughout the streaming session, with approximately 92% of records classified as Good and 8% as Moderate. No records reached Unhealthy or above during the observed windows.

Notable observations from the windowed classification output:

- California and Arizona consistently appeared in Moderate band windows — consistent with documented wildfire smoke and desert dust events in the Western US
- Midwest states (Iowa, Illinois, Indiana) predominantly appeared in Good band windows — consistent with lower industrial density
- The sliding window chart showed smoother band transitions compared to tumbling windows — confirming that 60-second overlapping windows reduce the impact of transient spikes
- No Very Unhealthy or Hazardous windows were observed — the dataset period did not include major air quality emergency events in the sampled cities

9. Marking Scheme

Assessment Criterion	Marks	Score
Task 1 — Replay producer publishes dataset to Kafka continuously with loop-back	4	
Task 2 — Streaming job parses JSON, applies windowed aggregations, classifies, writes to Redis	10	
Task 3 — Watermark correctly applied; late data handling explained in report	6	
Task 4 — Dashboard auto-refreshes with live data across all required sections	8	
Analysis — Both questions answered with clear reasoning and dataset-specific insight	2	
TOTAL	30	
BONUS — Sliding window query + side-by-side dashboard comparison	+3	